Failure (a crashing tool, and a bad description) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `tool_selection_practice.py` (Failure section): a crashing tool and a vague description are both real production bugs you will cause yourself at least once — better to see them here, on purpose, where nothing's at stake. **If not:** the Build Task's "one tool designed to sometimes fail" would be the first time you ever turned a raised exception into a clean, model-readable error string.

Basic Version

Approach 1 — both halves, the direct way

```
# tool_selection_practice.py — Failure section
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

# --- Half 1: a crashing tool, made safe ---

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"

model_with_tools = ChatOpenAI().bind_tools([flaky_divide])
response = model_with_tools.invoke("Use the tool to divide 10 by 0")

for call in response.tool_calls:
    result = flaky_divide.invoke(call["args"])
    print("Tool result:", result)  # "Error: division by zero", not a crash

# --- Half 2: a vague description, sharpened ---
```

```
@tool
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"

@tool
def precise_weather(city: str) -> str:
    """Get the current weather (temperature, conditions) for a named city."""
    return f"Sunny in {city}"

def count_calls(weather_tool, runs: int = 5) -> int:
    model_with_tools = ChatOpenAI().bind_tools([weather_tool])
    called = 0
    for _ in range(runs):
        response = model_with_tools.invoke("what's the weather like")
        if response.tool_calls:
            called += 1
    return called

vague_hits = count_calls(vague_weather)
precise_hits = count_calls(precise_weather)
print(f"Vague description: called {vague_hits}/5 times")
print(f"Precise description: called {precise_hits}/5 times")
```

Expected output (example):

```
Tool result: Error: division by zero
Vague description: called 1/5 times
Precise description: called 5/5 times
```

This shows both halves working: an error string instead of a crash, and a measurable difference between a vague and a precise description. It reruns the whole trial from scratch for each half rather than reusing the ambiguity exercise's trial function — fine here, but real code should share that logic instead of duplicating it.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — the error sent back to the model, and a fair description test

```
# tool_selection_practice.py — Failure section
from langchain_core.tools import BaseTool, tool
from langchain_openai import ChatOpenAI


# --- Half 1: a crashing tool, made safe, and the error sent back ---

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"


def demonstrate_safe_failure() -> None:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([flaky_divide])
    # how: the same growing message list as Doc04's memory exercise
    messages = [{"role": "user", "content": "Use the tool to divide 10 by 0"}]
    response = model_with_tools.invoke(messages)
    if not response.tool_calls:
        print(f"No tool called: {response.content}")
        return

    # how: the model's tool request goes into the history first
    messages.append(response)
    for call in response.tool_calls:
        result = flaky_divide.invoke(call["args"])
        print(f"Tool call {call['args']} -> {result}")
        # why: tool_call_id ties this result to the exact request, so the
        # model knows which of its calls this answer belongs to
        messages.append(
            {"role": "tool", "content": result, "tool_call_id": call["id"]}
        )
```

```
    # why: a second call, now with the error in the history — this is
    # the model really reacting to the tool's result
    final = model_with_tools.invoke(messages)
    print(f"Model's reaction to the error: {final.content}")


# --- Half 2: a vague description, sharpened ---
# how: @tool("get_weather") gives both tools the same name, so the
# description is the only thing that differs between the two trials

@tool("get_weather")
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"


@tool("get_weather")
def precise_weather(city: str) -> str:
    """Get the current weather (temperature, conditions) for a named city."""
    return f"Sunny in {city}"


# how: BaseTool is the type every @tool function has — so this hint
# says "weather_tool is a tool", the same way `prompt: str` says text
def count_tool_calls(weather_tool: BaseTool, prompt: str, runs: int = 5) -> int:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([weather_tool])
    called = 0
    for run_number in range(runs):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            called = called + 1
    return called


def demonstrate_description_fix() -> None:
    prompt = "what's the weather like"
    vague_hits = count_tool_calls(vague_weather, prompt)
    precise_hits = count_tool_calls(precise_weather, prompt)
    print(f"Vague description: called {vague_hits}/5 times")
    print(f"Precise description: called {precise_hits}/5 times")


if __name__ == "__main__":
    demonstrate_safe_failure()
```

```
    print()
    demonstrate_description_fix()
```

Expected output (example):

```
Tool call {'a': 10, 'b': 0} -> Error: division by zero
Model's reaction to the error: Dividing by zero isn't possible.

Vague description: called 1/5 times
Precise description: called 5/5 times
```

Difference from Basic: the error string is really sent back to the model — the request goes into `messages`, then a `{"role": "tool", ...}` message tagged with the call's `tool_call_id`, then a second call — so the reaction you print is the model reacting to the actual error (the "Getting a tool's failure back to the model" topic). And `@tool("get_weather")` gives both tools the same name, so the description is the *only* thing that differs between the two trials — in Basic, the names differ too (`vague_weather`, `precise_weather`), and a tool's name is also a hint to the model. `count_tool_calls(weather_tool: BaseTool, ...)` also types its tool argument: `BaseTool` is the type every `@tool` function has — the Build Task's harness uses `list[BaseTool]` the same way.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — one function body, two descriptions, from a factory

Story — `make_weather_tool(description)`: Approach 1 writes two separate functions with `@tool("get_weather")`. A factory builds both from ONE function body: it sets the docstring (`__doc__`), then wraps the function with `tool(...)` — which is exactly what `@tool` does. **If not:** the two tool bodies could drift apart, and a different result might come from the code, not the description.

```
# tool_selection_practice.py — Failure section
from langchain_core.tools import BaseTool, tool
from langchain_openai import ChatOpenAI


def make_weather_tool(description: str) -> BaseTool:
    # why: ONE function body for both trials — only the description
    # changes, so any difference in the split comes from the wording
    def get_weather(city: str) -> str:
        return f"Sunny in {city}"

    # how: @tool reads the docstring as the description, so set it first,
    # then wrap the function with tool(...) — the same as writing @tool
    get_weather.__doc__ = description
    return tool(get_weather)


def count_tool_calls(weather_tool: BaseTool, prompt: str, runs: int = 5) -> int:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([weather_tool])
    called = 0
    for run_number in range(runs):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            called = called + 1
    return called


def demonstrate_description_fix() -> None:
    prompt = "what's the weather like"
    vague_tool = make_weather_tool("gets data")
    precise_tool = make_weather_tool(
        "Get the current weather (temperature, conditions) for a named city."
    )
```

```
    vague_hits = count_tool_calls(vague_tool, prompt)
    precise_hits = count_tool_calls(precise_tool, prompt)
    print(f"names: {vague_tool.name}, {precise_tool.name}")
    print(f"Vague description: called {vague_hits}/5 times")
    print(f"Precise description: called {precise_hits}/5 times")


if __name__ == "__main__":
    demonstrate_description_fix()
```

Expected output (example):

```
names: get_weather, get_weather
Vague description: called 1/5 times
Precise description: called 5/5 times
```

Difference from Approach 1: both tools come from the same body and get the same name (`get_weather` , the function's own name), so the description is guaranteed to be the only difference.

Which one should you actually write? Intermediate's version covers what this exercise asks for — a caught error instead of a crash, and a measured before/after description comparison. Two things are worth naming as gaps rather than fixing here: a bare `except Exception` treats a real bug in the tool's own code the same as an expected failure like a divide-by-zero, and a 5-run split is a quick sanity check, not something to act on without a bigger sample. Both are worth reaching for on a tool you're about to keep and reuse for real, not for this exercise.